The Minions turn-lattice player

System writeup · 10 October 2026 · champion gen 660 of the `lattice537` lineage · weights in `weights/minions/lattice-gen660`

> **In one paragraph.** The player searches in two tiers. Tier 1 is AlphaZero-style MCTS over *complete turns*, using progressive widening. Tier 2 is a second MCTS over the *lattice of partial turns*: sets of commitments ordered by inclusion. Tier 2 proposes the turns that tier 1 compares. Two small networks drive both tiers:
>
> - a position trunk with a value head, 1.40M parameters;
> - a partial-turn head, 1.03M parameters.
>
> The current lineage started on 4 October from the Athenan steps net gen 537. Gen 660 has been champion since 9 October, promoted at 0.562 over gen 556. With the search fix of 10 October, the user rates it at roughly their own level in the UI. The current training run changes three things in self-play: random handicaps, anchored turn choice, and games from the start position. Since 20:15 on 10 October, the champion plays all self-play games. No net trained since the 10 October restart has beaten gen 660 yet.

1. The problem: one turn is a combinatorial action

Minions is a two-player game on a 49-cell hex board, stored as a 7×7 grid. A player wins by killing the enemy necromancer, or on net worth at the 30-turn cap. Each turn has three parts:

- attacks, which decide the fate of each enemy unit;
- movement, which gives each friendly unit a destination;
- spawns, which buy units onto spawn cells with money.

The legal turns are the product of these choices, subject to a realizability constraint: some legal order of moves and attacks must achieve the declared outcome. That leaves far too many turns to enumerate, and their legality is expensive to check. The design gets around both problems by searching over *declarations* instead of move sequences.

The first player gets a money handicap (the bid). Fixed at 2 in older self-play, it is now drawn at random per game.

2. The turn lattice

A turn is a set of **commitments**:

- each enemy unit gets a fate (stays, killed, or unsummoned);
- each friendly unit gets a destination;
- each spawn cell gets a unit or nothing.

A **lattice node** is a partial set of commitments that can still be completed to a legal turn. Nodes are ordered by inclusion. The source is the empty declaration, the sinks are complete turns, and every edge adds one commitment. There is no fixed variable order: a node's children are every feasible (cell, choice) pair for every undecided variable of the current phase.

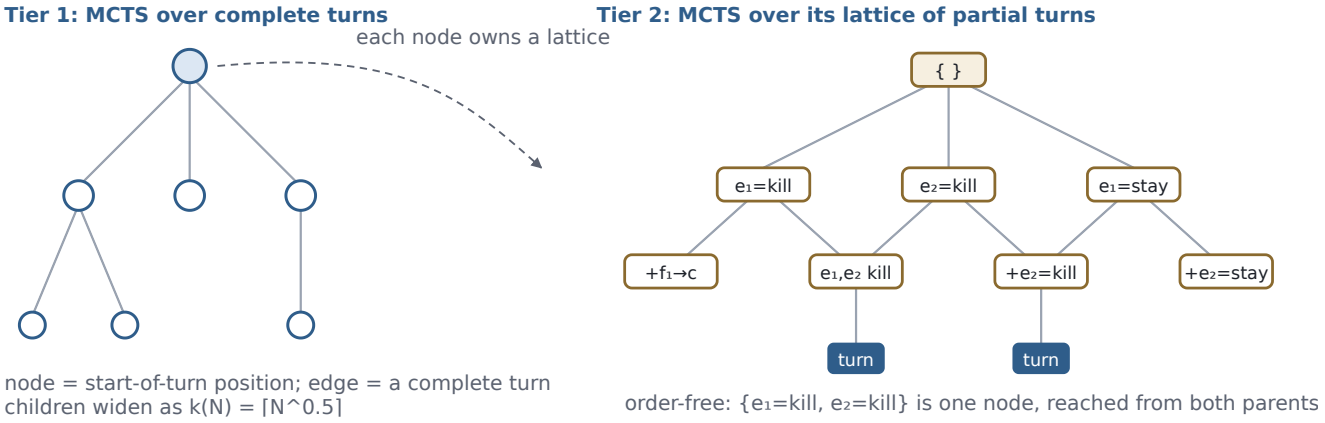


Figure 1. Two tiers. A tier-1 widening is one tier-2 descent from the lattice source to a sink, i.e. a complete turn. Lattice nodes are partial declarations ordered by inclusion, so different commitment orders reach the same node.

- **Phases.** Fates come first, until every enemy is fated. Then moves, until every friendly unit has a destination. The attack part is then realized. Spawns run last, on the real spawn-phase board, until none remain available. A spawn cell whose only remaining option is "nothing" is committed automatically.
- **Necromancer.** Its fate is never a choice. If the enemy necromancer can be killed this turn, the tier-1 node is terminal with that score. A complete turn after which the opponent can kill our necromancer is scored as a loss on the spot.
- **Keys and inclusion.** A node is a `mask`, one bit per variable, plus 6 bits per variable for the choice. Node X includes node P iff `P.mask & !X.mask == 0` and `(X.packed ^ P.packed) & field(P.mask) == 0`. That is a handful of word operations.
- **Feasibility.** Every offered child carries a *witness*, a concrete completion that proves it realizable. The checks are layered from cheap to expensive:

  1. cheap filters;
  2. a fast realizer;
  3. a Hall-type assignment bound;
  4. an exact search.

  Witnesses are checked lazily: a child's check runs when the child is first expanded. Each lattice keeps a pool of witnesses and a list of *nogoods*, keys proven infeasible, which prunes the nodes that include them. When a widening exceeds 1000 expansions, an exact SAT encoding of the attack phase takes over (CaDiCaL: 2.7 ms to encode, 0.065 ms per query). It either returns a witness or refutes the node.
- **Cyclic moves.** Rotations of friendly units are legal, as in the reference rules since 8 October. The lattice can therefore hold distinct declarations that reach the same position. Tier 1 merges them by position: a turn that reaches a position an existing child already holds is a repeat, not a new child.

3. Tier 2: PUCT over the lattice

- **Selection.** From the source, take the child with the highest PUCT score ($c = 1.5$), expanding along the way, until a sink. Every descent is a whole source-to-turn path. Priors are a softmax within each cell, so each cell's choices sum to 1. A single PUCT comparison over every (cell, choice) pair therefore picks the cell and its choice at once.
- **Expansion** is one tier-2 net call. It returns, for each undecided cell:

    - prior logits over the cell's choices;
    - one value per child;
    - a value for the node itself.

  The expansion counts as direct visits at those values.
- **Sinks** are valued by the tier-1 value of the resulting position, flipped to the mover, or by their terminal score.
- **Lazy sublattice propagation.** A visit to a node counts as a visit to every node it includes. There is no path backup. The search keeps a list of direct visits and computes $N(P) = \Sigma_{X \supseteq P}\, n_X$ and $W(P)$ with one scan of that list per selection step, which credits each child of P at the same time. Lazy and brute-force statistics agree in tests.
- **Targets average complete turns only** (the user's rule). The lattice's own predicted values only steer the search. Every training value comes from sink returns: complete turns valued by tier 1.

4. Tier 1: AlphaZero over turns with progressive widening

- A tier-1 node is a start-of-turn position, and its children are the complete turns found so far. A visit **widens** while the node has fewer than $k(N) = \lceil N^{0.5} \rceil$ children. A widening is one tier-2 descent in that node's lattice. A repeat is retried up to 20 times, after which the node stops widening.
- PUCT with $c = 1.5$ and mean backup, with the side flipped at every edge.
- **The tier-1 prior is effectively uniform:** softmax(child value / 100). The original prior, log-odds accumulated along the lattice path, lost to a uniform prior 0.427 over 384 games. Value-based priors at temperatures 0.05–0.6 did no better than uniform. No learned tier-1 prior has helped yet.
- **Visits in flight.** Each search keeps several visits running at once, with virtual loss along the path, so the net is called in batches.

> **The in-flight widening race** (found and fixed 10 October, commit `9cbde00`). While a node's widening was in flight, it had fewer children than a serial search would give it. The other in-flight visits then piled onto whichever children existed, whatever their value. Every self-play generation before gen 754 ran with 64 visits in flight.
>
> The fix has two parts:
>
> - a visit waits while its node has a widening pending and fewer than $k(N)$ children;
> - the root, which has no virtual loss, counts the other in-flight visits in its widening threshold.

turn-3 test position, 400 visits	serial	16 in flight	64 in flight
visits to turns with Q < 0.3, before the fix	6	22	79
after the fix	6	8	19

A strength check at 200 visits, 32 games per setting against serial, found no trend: ×4 0.469, ×8 0.438, ×16 0.531. The fix itself was checked in detail on one position only.

5. Networks

net	architecture	input	output
tier-1 trunk + value	`pos:96x8`: 96 channels, 8 residual blocks, 1.40M params	the full-turn position only: unit and graveyard planes; money, reinforcement and turns-left scalars; a constant plane	value, plus a 96×7×7 trunk image (fp16), cached per position
tier-2 head	`96x6`, 1.03M params	the trunk image plus the *lattice view*: the original position; enemy fates; decided friends at their origin; final friendly units at their final cell; a from-spawn bit; projected scalars	46 slots per cell (3 fates, 37 move destinations, 6 spawns) as prior logits and child values; node value; position value; energy

- **One value function** (`--t1-node`). The exported tier-1 value is the tier-2 head's node value at the empty declaration. Tier 1 and tier 2 therefore cannot disagree about a turn start.
- **Partial turns never enter the trunk** (the user's rule). The trunk sees full-turn positions; decisions reach tier 2 only. Tier 1 trains on turn records and tier 2 on lattice records.
- An **energy head** (a softmax over a node's tier-1 children) is trained with weight 0.2. The search does not use it.
- **Runtime.** Two ONNX graphs run under TensorRT; the CUDA execution provider returns wrong values for some shapes. A single runner merges requests from all games into large batches. Tier-2 batches gather cached trunk images instead of recomputing the trunk.

6. Self-play: the current recipe (gens 759+)

setting	value
self-play net	the champion (`SP_CHAMP=1`), currently gen 660
concurrency	16 workers × 16 games, 8 tier-1 visits in flight per search, games started over a 60 s ramp; ~7 GB RSS
starting position	the initial position; no opening pool or random opening (`--from-start`)
handicap	uniform per game on −5..10 dollars (`--random-handicap`)
tier-1 search	800 visits per turn; prior softmax(value/100); c 1.5; widening $\lceil N^{0.5} \rceil$
turn choice	anchored (below): temperature 1, Dirichlet α 0.3, weight 0.25
time split	400 s of self-play, then up to 190 s for the tier-2 phase

Anchored turn choice

1. Run the tier-1 search, then pick a turn from the root visits with temperature and Dirichlet noise.
2. Take a random lattice path to that turn and pick one node on it as the *anchor*.
3. Run a from-scratch tier-2 search rooted at the anchor, in a fresh lattice. Its record is a tier-2 training target.
4. Draw the played turn from the complete turns that search reached, by descents, with temperature and Dirichlet noise.

The other nodes on the path join the tier-2 phase queue with probability 0.15. This lets the game leave the top tier-1 turns, toward turns that share a reasonable partial declaration with

them, without a uniformly random opening.

Records

- **Tier-1 value targets:** the root of each searched turn, plus every tier-1 node with at least 16 visits, at the mean of its backed-up tier-1 values. That is about 26 records per searched turn.
- **Tier-2 targets** come from rooted searches: the anchors plus the tier-2 phase. Each search runs 128 descents, visiting every kid once first. A kid's value target is the best complete turn reached below it. The prior target is a per-cell softmax of those values at $\tau = 0.02$.
- **Energy groups:** each turn start with its tier-1 children's declarations and visits.
- Game results are never trained on.

per generation (gen 758, gen-660 self-play)	count
games / searched turns in 400 s	267 / 1,024
tier-1 records	26,892
tier-2 records: anchors / tier-2 phase	1,005 / 3,002
tier-2 phase throughput (finished in 54 s of its 190 s)	56 searches/s, 55k expansions/s

7. Training and gating

- **Learner** (`python/minions_lattice_learner.py`). Joint training:
  - tier-2 prior cross-entropy;
  - child-value and node-value BCE;
  - tier-1 value BCE, on 256 tier-1 records per step;
  - energy loss at weight 0.2.

  The tier-2 losses also train the trunk. AdamW at learning rate 10^{-3}, one epoch over a window of the last 8 generations, warm-started from the previous net in the chain. About 130 steps and 60 s per generation.
- **Cycle** (`data/models/minions/lattice537/cycle.sh`):
  - Net G−1 trains while generation G self-plays, about 8 minutes per generation.
  - Training continues a *chain*: each net starts from the previous one.
  - The champion plays the self-play games.
  - After a restart, no training runs until the window holds 4 generations of data (`MIN_WINDOW`).
- **Gate**, every 4 generations: 64 seat-swapped pairs (128 games) against the champion, at 100 tier-1 visits with 16 in flight. Each pair starts after 2 net-sampled opening turns at handicap 3, the calibrated even point. A candidate becomes champion at ≥ 0.55 (standard error ≈ 0.044).

8. How it got here

date	change	result
10-03	design (the user's); built and trained from random init	shelved at gen 76: lost 1–31 to the heuristic greedy bot (the end-heavy curriculum taught it not to spawn)
10-04	`lattice537`: tier 1 from Athenan gen 537; "bridge" self-play on its value net (gens 13–45)	reached Athenan's level, not past it
10-05	gating; tier 2 distilled from Athenan (D0)	D0 beat Athenan-537 0.594: the first time the full design did
10-06	single value function, 64×6 trunk; value-based tier-2 targets; lattice view	p64: 0.672 vs Athenan-537
10-07	chained training; uniform tier-1 prior; exact SAT check	champions 338 → 424
10-08	rooted tier-2 phase; 800 visits; nets grown to 96×8 / 96×6; cyclic-move rules	prior CE excess 0.32 → 0.19; champions 478, 496, 534, 556
10-09	champion 660 (0.562 vs 556); stepwise self-play tried	gen 648 vs 526: 0.555 ± 0.020 over 640 games. Stepwise lost: 0.49 → 0.27 vs 660
10-10	race fix; anchored turn choice; from-start; random handicap; champion self-play	post-restart nets 0.40–0.42 vs 660 (§9)

9. The 10 October restart

All three recipe changes started from champion 660 at gen 754. Every earlier generation had trained on data from the race-biased search.

net	trained on	vs 660
gen 754	one generation of data (19k tier-1 / 3.7k tier-2 records)	0.234 ± 0.053
gen 755 / 756	two / three generations	0.141 / 0.164
reset to 660, with 4 generations of warm-up self-play by 660		
gen 757	4 generations, the first step from 660	0.281 ± 0.056
gen 758 / 760	self-play by the chain	0.422 / 0.398
gen 764	self-play by the chain	0.367
reset to 660 again; the champion plays self-play from gen 759 on		

As a control, gen 752 from the old chain scored 0.391 ± 0.061 in the new gate format, so the format itself is not what makes the new nets look weak.

10. Lessons

- **Visit counts are not prior targets in an order-free lattice.** A complete turn is a visit for *all* of its commitments, so the counts fail either way:
  - marginal counts reward leftover choices;
  - first-step counts are pushed by PUCT toward choices not yet in any found turn.

Both collapsed a good tier 2, to 0.14–0.25 and to 0–64. Only value-based per-cell softmax targets have worked.

- **Losses do not predict strength.** Stepwise targets improved every in-window loss while the gates fell from 0.49 to 0.27. Only matches count.
- **Parallelism can bias the search.** The widening race distorted visit distributions for hundreds of generations before anyone noticed.
- **Window size matters on a restart.** One generation of data cost 660 about 0.27 in score in a single step.
- **A fixed handicap leaves money unpriced.** Gen 660's value for yellow moved only from 0.52 to 0.47 across −30..+30 dollars. Equal-net matches are even near a handicap of 3.
- **Proposal quality bounds the search.** Tier 1 ranks the turns it is given well, but progressive widening only ever gives it $\sqrt{N}$ turns, all proposed by tier 2. A weak tier 2 means weak play.

11. Throughput and memory

- Self-play is bound by latency, not memory or GPU compute. Throughput tracks the total visits in flight, games × in-flight. About 2,048 in flight gives about 3 searched turns/s, whether that is 4 games × 32 or 16 games × 8.
- The single ONNX runner's per-call host cost dominates: 1–2 ms per call whatever the row count. The GPU is 16–40% busy.
- Memory is dominated by lattice nodes, about 25 per tier-1 visit. Changes on 10 October cut memory per node by about 45%:
  - a hashed key index;
  - witnesses stored as differences;
  - a packed node pool;
  - boxed rarely-used fields.

12. Open issues

1. **Max versus mean.** Rooted tier-2 value targets take the *best* complete turn below each kid. That conflicts with the rule that targets *average* complete turns. Unresolved.
2. **The tier-2 phase sits idle for about 70% of its budget** (54 of 190 s at gen 758). Moving that time to self-play, or raising the queue fraction, is free throughput.
3. **No post-restart net has beaten 660.** Champion self-play only started at gen 759. If the gates stay below 0.45, the next steps to try are a lower learning rate for the first generations, or mixing older data back in.
4. **Money is still unpriced.** Random-handicap data is meant to teach the value head about money. The post-opening even bid (`--even-bid`) waits until it can.
5. **The tier-1 prior is still uniform;** the energy head is trained but unused.
6. **The training-data explorer UI** (games, their records and how they were generated) is designed but not built.

Appendix: running it

```
source scripts/ort-env.sh            # TensorRT runtime; required

# play / inspect in the UI (lattice panel on :7107)
./target/release/minions-engine serve --addr 127.0.0.1:7105 \
    --lattice-net vprior=100:weights/minions/lattice-gen660

# match two lattice nets (gate format: 2 sampled opening turns, handicap 3)
./target/release/minions-engine steps-match --a vprior=100:<dirA> \
    --b vprior=100:<dirB> --pairs 64 --n1 100 --threads 16

# the training cycle (detached); stop with: touch data/models/minions/lattice537/STOP
SP_CHAMP=1 MIN_WINDOW=4 T1_ARCH=pos:96x8 T2_ARCH=96x6 RESET_MIN=0 \
GEN_FLAGS="--workers 16 --games 16 --inflight 8 --ramp-secs 60 --anchor --from-start --
random-handicap" \
setsid nohup bash data/models/minions/lattice537/cycle.sh >>
data/models/minions/lattice537/cycle.log 2>&1 < /dev/null &
```

code	role
`crates/games/minions/src/lattice.rs`	lattice, tier-2 PUCT, lazy propagation, rooted searches
`tier1.rs`	tier-1 MCTS with progressive widening, in-flight handling
`latgen.rs`	self-play driver (`lattice-gen`), anchored choice, records
`lattice_ort.rs` , `lview.rs` , `sat.rs`	ONNX/TensorRT evaluator; lattice view; SAT check
`stepsp.rs` , `lattice_ui.rs`	matches and openings; the UI's lattice panel server
`python/minions_lattice_learner.py`	learner and ONNX export
`docs/minions-lattice.md`	the design document and its history